

4

Дискретни структури

доц. д-р Кристина Близнакова
доц. д-р Владимир Николов

2021-2022 , ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - ВАРНА

Съдържание

5.1. Граматика. Дефиниции.

5.2. Съставяне на граматика.

5.3. Класификация на граматиките.

5.4. Бакус – Наур форма за представяне на синтаксиса.

Обобщение

Азбука - непразно крайно множество от символи ,
означаваме най-често със Σ . Елементите на Σ се наричат
букви.

Пример: $\Sigma = \{a, b\}$

**Низ или дума от Σ е всяка крайна последователност от
буквите на Σ .**

Пример: ако $\Sigma = \{0, 1\}$, то **0101** е низ на
азбуката Σ .

Обобщение

Σ^* = множество от всички възможни думи над азбуката Σ ,
включително и празния низ

Σ^+ = множество от всички възможни думи над азбуката Σ ,
без празния низ

Формален език

Формален език - бележим с **L**

L е множество от низове с крайна дължина, извлечено от крайна азбука Σ :

$$\mathbf{L} \subseteq \mathbf{\Sigma^*}$$

Формални езици

Формалните езици се задават чрез:

- ☐ изреждане на низовете : $\{aa, bb, ab, \dots\}$;
- ☐ низове, генерирани от някоя формална граматика;
- ☐ низове, генерирани от регулярен израз;
- ☐ низове, разпознавани от автомати (Тюринг машина или краен автомат).

Формална граматика

G – граматика – множество, състоящо се от 4 множества:

$$G = \{\Sigma, N, P, S\}$$

Σ – азбука или множество от **букви** на езика L , описван от граматиката, които символи наричаме още **терминални**.
Означаваме с малки букви.

N – крайно непразно множество от символи (**променливи**), наречени **нетерминални**. Означаваме с големи букви.

P – крайно множество от **правила**, определящи как се генерират думите на езика.

S – начална **променлива**, **начален символ**. От него започва генерирането на елементите на езика. $S \in N$

$$N \cap \Sigma = \emptyset \quad - N \text{ и } \Sigma \text{ нямат общи членове.}$$

Дефиниции

- **Нетерминал** – символи (**променливи**) от граматиката, които могат да бъдат заместени (разширени) с други символи и/или думи.
- **Терминал** - елементи от езика, които **не могат да бъдат заместени с никакви други символи(думи)**. След получаване на терминален символ не може да бъде приложено следващо правило.
- **Правило (продукция)** – правило от граматиката за заместване на символи.

➤ Правилото има вида: $\alpha \rightarrow \beta$

Дефиниции

Ако S е началният символ за дадена граматика, то тогава трябва да има поне една продукция от вида:

$$S \rightarrow \beta$$

Казваме замести S с β .

Пример

Нека $\Sigma = \{a, b, c\}$

Да съставим граматиката G на езика $L \subseteq \Sigma^*$.

Нека граматиката да е дефинирана чрез следното множество правила P :

$S \rightarrow \Lambda$

$S \rightarrow aS$

$S \rightarrow bS$

$S \rightarrow cS$

Как да разберем, че тази граматика описва L ?

Като използваме правилата на граматиката трябва да можем да опишем всеки един низ от езика описан с G .

Нека да проверим дали думата **aacb** принадлежи на L ?

Пример

Правилата на граматиката на езика A^* :

$\Sigma = \{a, b, c\}$.

Нека да проверим дали думата **aacb** принадлежи на

$L \subseteq \Sigma^*$?

$S \rightarrow \Lambda$

$S \rightarrow aS$

$S \rightarrow bS$

$S \rightarrow cS$

- ✓ Започваме с начален символ S
- ✓ Нека да изберем продукцията $S \rightarrow aS$.
Ще заместим S с дясната част на продукцията.
- ✓ Нека да изберем отново същата продукция $S \rightarrow aS$.
И да заместим S с дясната част на продукцията.
- ✓ Нека да изберем продукцията $S \rightarrow cS$.
Ще заместим S с дясната част на тази продукция.
- ✓ Ще изберем продукцията $S \rightarrow bS$.
Ще заместим S с дясната част на тази продукция.
- ✓ Най-накрая ще изберем продукцията $S \rightarrow \Lambda$
Ще заместим S с дясната част на тази продукция.

✓ S
↓
✓ aS
↓
✓ aaS
↓
✓ $aacS$
↓
✓ $aacbS$
↓
✓ $aacb$

Съдържание

5.1. Граматика. Дефиниции.

5.2. Съставяне на граматика.

5.3. Класификация на граматиките.

5.4. Бакус – Наур форма за представяне на синтаксиса.

Съставяне Граматика

Когато се опитваме да намерим граматиката на даден език, трябва първо да отговорим на въпроса, какъв е езикът?

Краен

Безкраен

Краен/безкраен език

Ако езикът е краен – граматиката може да съдържа всички продукции от типа

$$S \rightarrow w$$

Например езика $\{a, ba\}$

$$S \rightarrow a \mid ba$$

При безкрайния език - рекурсия

Пример

Определете N и Σ за граматиката L , генериране със следните параметри:

$$p = \begin{cases} S \rightarrow aSB \\ S \rightarrow aB \\ B \rightarrow b \end{cases}$$

Пример

Намерете граматиката за $\{\Lambda, bb, bbbb, bbbbbb, \dots\}$

..... $(b)^{2n}$

$S \rightarrow bbS$

n - пъти го прилагаме $(b)^{2n} S$

$S \rightarrow E$

$E \rightarrow \Lambda$

}

1 път ги прилагаме $(b)^{2n}$

Пример

Намерете граматиката за $\{bb, bab, baab, baaab, \dots\}$

..... $ba^n b, \quad n \geq 0$

✓ $S \rightarrow bEb$

✓ $E \rightarrow \Lambda$

✓ $E \rightarrow aE$

Съдържание

5.1. Граматика. Дефиниции.

5.2. Съставяне на граматика.

5.3. Класификация на граматиките.

5.4. Бакус – Наур форма за представяне на синтаксиса.

Граматики

За какво използваме?

Граматиките **генерират** думи.

Машините **приемат/разпознават** думи.



Система за описване на граматика

- Синтаксисът е структурата на един език. През 1950 Ноам Чомски развива идеята за ***контекстно-свободни граматики***.
- John Backus и Peter Naur разработват нотационна система за описване на тези граматики, наречена Backus-Naur форма (БНФ) – за първи път използвана да опише синтаксиса на езика Algol60.

Всеки компютърен специалист трябва да знае как да чете, интерпретира и прилага БНФ описанията на синтаксиса на програмния език.

Класификация и йерархия на формалните граматики. Йерархия на Ноам Чомски

Американският лингвист Ноам Чомски е извършил солидни изследвания върху формални граматики. Граматиките могат да бъдат класифицирани съгласно наложени ограничения към правилата:

- естествен език (**G0**);
- контекстно -зависима граматика (**G1**);
- контекстно-свободна граматика (**G2**);
- регулярна (линейна) граматика (**G3**).

Класификация и йерархия на формалните граматики.

Тип 0 - най-обща

Тип 3 - най-рестриктивна

Ако граматиката има повече ограничения към правилата, тя е по-лесна за описание и реализация, но е с по-малка мощност, т.е. поражда по-малко низове в езика.

Класификация и йерархия на формалните граматики.

Тип 0: без ограничения.

Този тип граматики са най-общи. Правилата са във вида

$$\alpha \rightarrow \beta$$

където $\alpha \in (N \cup \Sigma)^+$, $\beta \in (N \cup \Sigma)^*$.

Няма ограничения към правилата, освен че в лявата страна на правило **не може да има празен низ**.

Тип 0 включва всички формални граматики. Дефинираните чрез тях езици са точно тези, които могат да бъдат разпознати от една машина на Тюринг.

Класификация и йерархия на формалните граматики

Тип1: контекстно-зависими граматики.

Правилата на тази граматика са във вида

$$\alpha \rightarrow \beta$$

където $\alpha = aAb$ и $\beta = awb$

a и b са произволни низове от символи от S (вкл терминални и нетерминални, празен). **w не може да е празна дума.**

w е непразен низ от символи от терминални и нетерминални символи.

$$w \in (N \cup \Sigma)^+$$

A е единствен нетерминал. Това означава, че **A** може да бъде заместен от **w** , само ако е обграден от символи **a** и **b** . (т.е. в конкретен контекст).

Под контекст на символ се разбира конкатенация на символ с други (терминални и/или нетерминални) символи.

Пример

$S \rightarrow cAd$

$Ac \rightarrow a$

$A \rightarrow ab$

Символите a, b, c, d са терминални

S и A - нетерминални

Класификация и йерархия на формалните граматики

Тип 2: контекстно-свободни граматики

Правилата са във вида

$$\alpha \rightarrow \beta$$

където α е единствен нетерминален символ, а β е произволен низ от символи (терминални и нетерминални):

$$\alpha \in N, \quad \beta \in (N + \Sigma)^+$$

Навсякъде, където се среща α той може да бъде заместен с β (независимо от контекста).

Контекстно свободна

Пример за контекстно
свободни от вида:

$$L = \{a^n b^n \mid n \geq 1\}$$

$S \rightarrow aSb$

$S \rightarrow ab$

Контекстно свободна

Пример за контекстно
свободни от вида:

$$L = \{X^n W Y^n \mid n \geq 0, W \in \Sigma^*\}$$

$$L = \{a^n b^n \mid n \geq 0\}$$

$$S \rightarrow \Lambda$$

$$S \rightarrow A$$

$$A \rightarrow ab$$

$$A \rightarrow aAb$$

Тъй като езикът L съдържа празната дума, то правилото $S \rightarrow \Lambda$ трябва да съществува. Това означава, че S **не трябва** да се появява никъде в дясната част, на което и да е правило от дадената граматика.

Класификация и йерархия на формалните граматики

Тип 3: Регулярна граматика

Граматика, която се състои от следните правила: в **лявата част**, на които има само един нетерминален символ, а в **дясната** конкатенация на терминален с нетерминален символ, само терминален символ или празен символ:

$$\alpha \rightarrow \beta$$

α – нетерминален символ

$$\beta \in (\Sigma^* \cup \Sigma^* N)$$

т.е. β е терминален символ, конкатенация на терминален символ с нетерминален или празна дума.

- ✓ Тези граматики са най-ограничени по отношение на брой породени символни низове.
- ✓ Езикът генериран чрез регулярните граматики се нарича **регулярен**.

Пример

$$L = \{a^i b^i c^i \mid i \geq 1\}$$

$$S \rightarrow aSBC$$

$$S \rightarrow abC$$

$$CB \rightarrow BC$$

$$bB \rightarrow bb$$

$$bC \rightarrow bc$$

$$cC \rightarrow cc$$

Тип 1

$$L = \{a^i b^k c^{ik} \mid i, k \geq 1\}$$

$$S \rightarrow aS$$

$$S \rightarrow aE$$

$$E \rightarrow bEc$$

$$E \rightarrow bc$$

Тип 2

$$L = \{a^i b^k c^m \mid i, m, k \geq 1\}$$

$$S \rightarrow aS$$

$$S \rightarrow aA$$

$$A \rightarrow bA$$

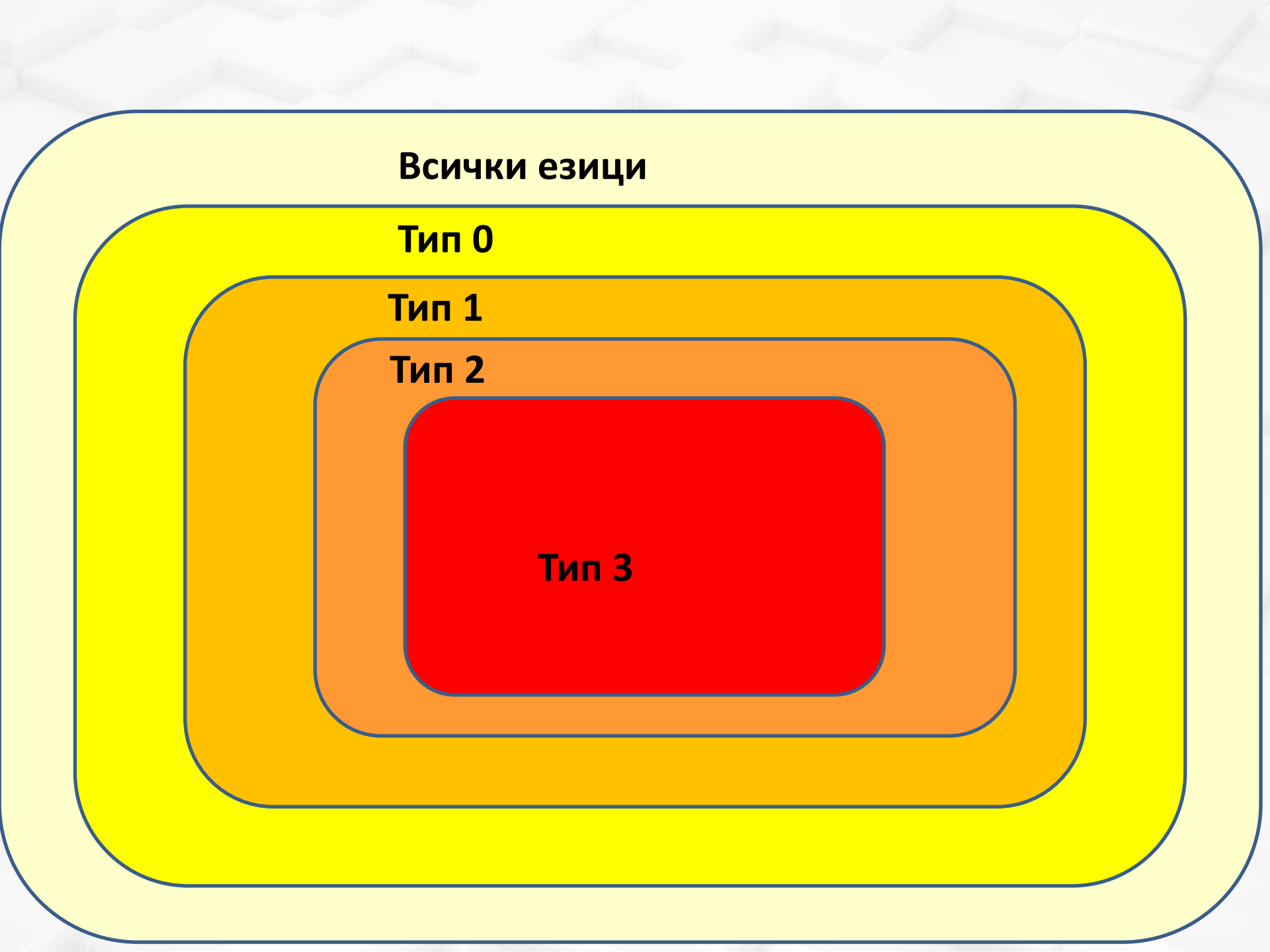
$$A \rightarrow bB$$

$$B \rightarrow cB$$

$$B \rightarrow c$$

Тип 3

Всички езици



Тип 0

Тип 1

Тип 2

Тип 3

Пример

Описване на изречение от формален език чрез използване на граматика. Използването на граматика помага, когато разглеждаме два вида проблеми, които възникват в приложения свързани с програмирането.

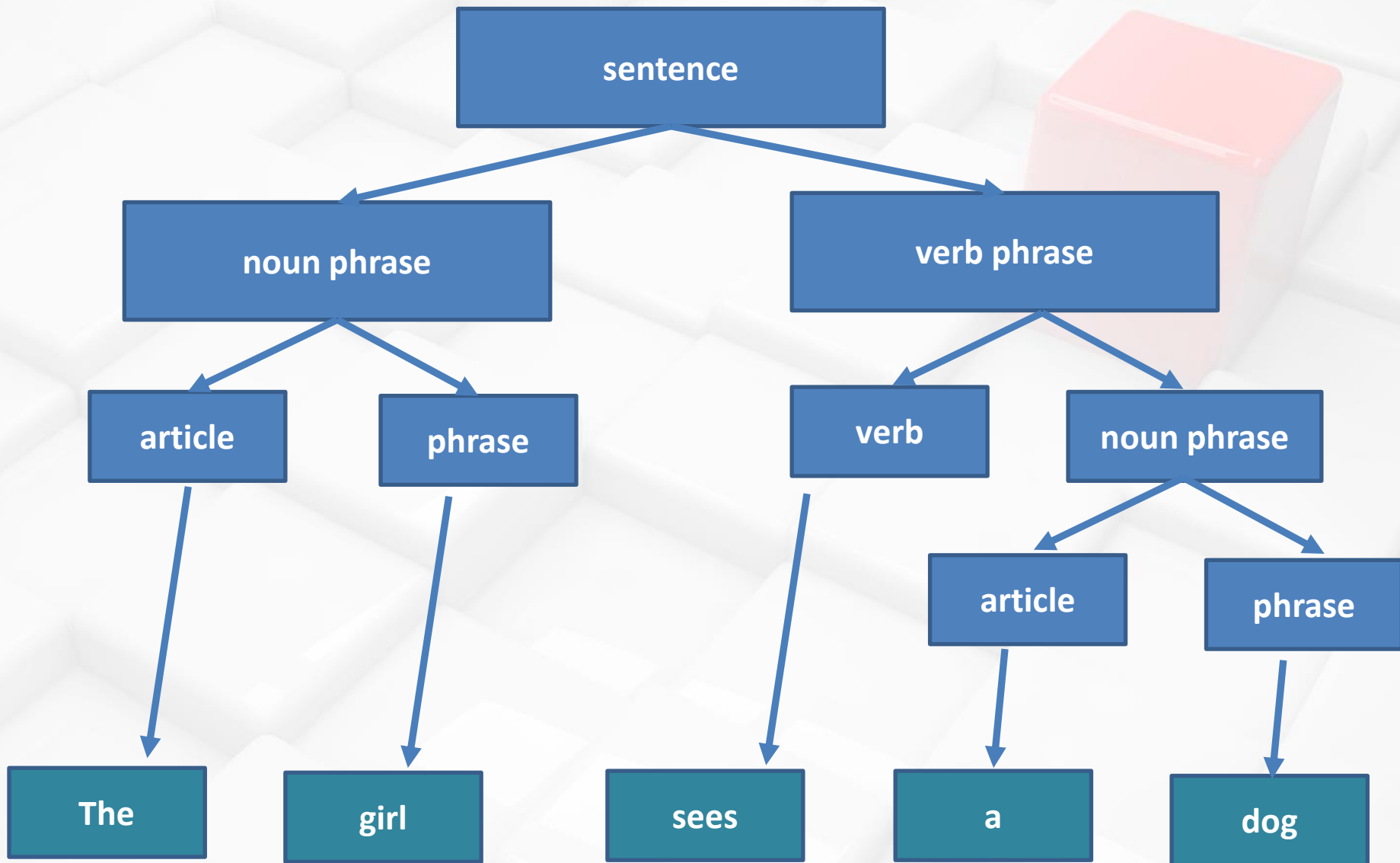
- (1) Как можем да определим дали комбинацията от думи е валидно изречение в един формален език?
- (2) Как да генерираме валидни изречения във формален език?

Пример от граматика, която генерира подмножество на английския език: това подмножество на английския език е дефинирано чрез лист от правила, които описват как да се генерира валидно изречение.

Пример

1. изречение е направено от **подлог_фраза** последвана от **глаголна_фраза**;
2. **подлог_фраза** е направено от **определителен член** последвано от **прилагателно** последвано от **съществително**, или
3. **подлог_фраза** е направена от **определителен член** последвано от **съществително**;
4. **глаголна_фраза** е направена от **глагол** последвана от **наречие**, или
5. **глаголна_фраза** е направена от **глагол**;
6. **определителен член** е *a*, или
7. **определителен член** е *the*;
8. **прилагателно** е *large*, или
9. **прилагателно** е *hungry*;
10. **съществително** е *rabbit*, или
11. **съществително** е *mathematician*;
12. **глагол** е *eats*, или
13. **глагол** е *hops*;
14. **наречие** е *quickly*, или
15. **наречие** е *wildly*.

Пример



Пример

От тези правила можем да направим валидни изречения чрез използване на замени, дотогава докато повече правила не могат да се използват:

sentence -> noun_phrase verb_phrase

noun_phrase -> article adjective noun

verb_phrase -> verb adverb

noun_phrase = {rabbit, mathematician}

verb_phrase = {eats, hops}

article = {a, the}

adjective_phrase = {large, hungry }

verb_phrase = {wildly, quickly}

Пример

От тези правила можем да направим валидни изречения чрез използване на замени, дотогава докато повече правила не могат да се използват:

sentence -> noun_phrase verb_phrase

noun_phrase -> article adjective noun

verb_phrase -> verb adverb

the adjective noun verb adverb

the large noun verb adverb

the large rabbit verb adverb

the large rabbit hops adverb

the large rabbit hops quickly

Валидно изречение

Други валидни изречения са:

a hungry mathematician eats wildly

a large mathematician hops

the rabbit eats quickly, и т.н.

The quickly eats mathematician не е валидно изречение.

Видове продукции (правила)

В зависимост от структурата на рекурсивната продукция:

- двойно рекурсивна продукция

$$x \rightarrow \alpha x \beta$$

x се получава при добавяне на α и β отляво и отдясно на x .
 α и β са низове.

- ляво рекурсивна продукция

$$x \rightarrow x \beta$$

x се получава чрез добавяне на низ отдясно

- дясно рекурсивна продукция

$$x \rightarrow \alpha x$$

x се получава чрез добавяне на низ отляво

Пример

$A w \rightarrow A A w$

лява рекурсия

$A w \rightarrow A w A$

дясна рекурсия

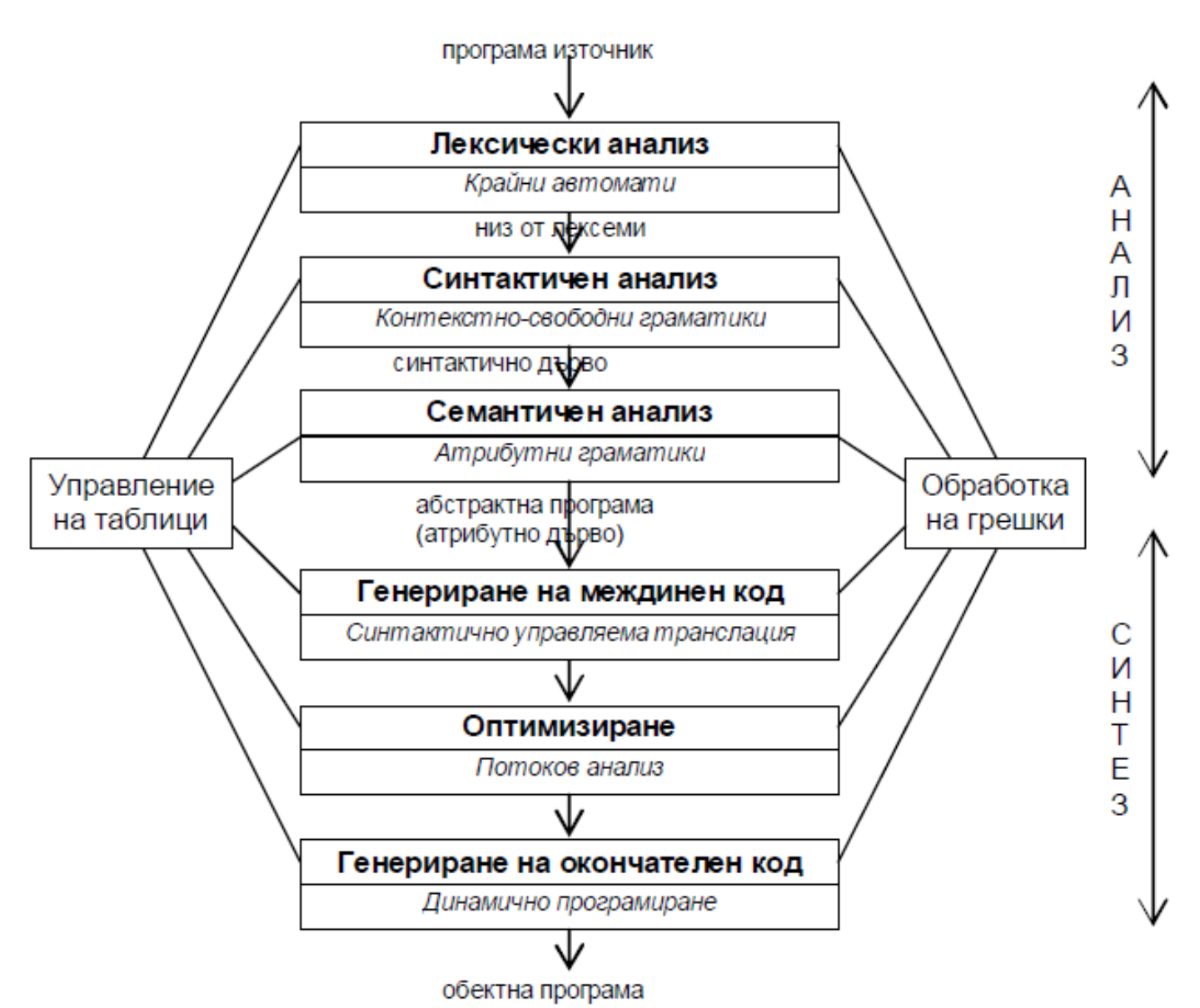
пример

Видове продукции (правила)

В зависимост от предназначението на продукциите, те се делят на :

- **Лексика** – правила за пораждане на думи;
- **Синтаксис** – правила за пораждане на изрази;
- **Семантика** – допълнителни правила, свързани със смисъла на породените думи или изрази.

В компютърните науки, най-често срещаният тип граматика е контекстно-свободна граматика. Този тип граматика могат да опишат рекурсивно синтактичната структура на много езика.



БНФ - Бакус-Наур форма

- ❑ За формализиране описанието на граматиките се използват т.н. синтактични метаезици: това са изкуствените езици, които се използват за описание на други езици (изкуствени или естествени).
- ❑ Съществуват метаезици за описание на **синтаксиса** и метаезици за описание на **семантиката**.
- ❑ Исторически, първият метаезик е формата (нотацията) на Бекус-Наур или т.н. БНФ, която е наречена така в чест на Бекус (главен разработчик на компилатора Фортран) и Наур (участник в разработката на АЛГОЛ-60). Първоначално БНФ е използвана за строго определяне на синтаксиса на езика Алгол.
- ❑ БНФ е начин за описание на програмни езици чрез формални граматики и правила.

Дефиниции

Граматиката на един език за програмиране, описан чрез БНФ, се състои от краен брой граматични правила, всяко с лява и дясна част, отделени със символ “::=”, вместо със стрелка “→”.

Символ “::=” се чете „по дефиниция е”.

Напр: $\alpha \rightarrow \beta$ се записва като $\alpha ::= \beta$ или $\alpha \Rightarrow \beta$

Лявата част на граматично правило се състои от металингвистична променлива – нетерминален символ, който представлява текст, заграден с металингвистични символи “{” и “}”.

Текстът в скобите е **име на синтактична категория** от езика, чиято роля в граматиката се изпълнява от нетерминалния символ.

Дясната част на граматичното правило се състои от една или повече думи от нетерминални и терминални символи – варианти (алтернативни дефиниции, алтернативни възможности). Те се отделят един от друг с металингвистичен символ вертикална черта “|”, чете се „или”.

Дефиниции

Конкатенация:

UV

Алтернатива:

$U \mid V$

Незадължителен елемент:

$[u]$

Помощни скоби:

(u)

Итерация: $u^n \mid n \geq 0$

u^* или $(u)^*$ или $\{u\}$

Итерация без празното

множество: $u^n \mid n > 0$

u^+ или uu^* или $(u)^+$ или $u\{u\}$

Пример

Запис в БНФ на цяло число:

$\langle \text{цяло число} \rangle ::=$

$\langle \text{цяло число без знак} \rangle \mid +\langle \text{цяло число без знак} \rangle \mid -\langle \text{цяло число без знак} \rangle$

$\langle \text{цяло число без знак} \rangle ::=$

$\langle \text{цифра} \rangle \mid \langle \text{цяло число без знак} \rangle \langle \text{цифра} \rangle$

$\langle \text{цифра} \rangle ::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

Пример

Генерирането на

$$A \rightarrow Aa, \quad A \rightarrow a, \quad A \rightarrow AB$$

се отбелязва с

$$A ::= Aa \mid a \mid AB$$

Пример C++

```
<type-specifier> ::= void  
    | char  
    | short  
    | int  
    | long  
    | float  
    | double  
    | signed  
    | unsigned  
    | <struct-or-union-specifier>  
    | <enum-specifier>  
    | <typedef-name>
```

Пример Java

<type> ::= <primitive type> | <reference type>

<primitive type> ::= <numeric type> | boolean

<numeric type> ::= <integral type> | <floating-point type>

<integral type> ::= byte | short | int | long | char

<floating-point type> ::= float | double

<reference type> ::= <class or interface type> | <array type>

<class or interface type> ::= <class type> | <interface type>

<class type> ::= <type name>

<interface type> ::= <type name>

<array type> ::= <type> []

Пример

Направете БНФ на генерирането на **signed integers** в десетична нотация.

Решение:

signed integer ::= *sign integer*

sign ::= + | -

integer ::= *digit* | *digit integer*

digit ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Пример

Операцията **if loop** в C/C++ има следната форма

```
if( израз )  
    { тяло }
```

$\langle \text{if_loop} \rangle ::= \langle \text{if_part} \rangle$

$\langle \text{if_part} \rangle ::= \text{if}(\langle \text{израз} \rangle) \{ \langle \text{тяло} \rangle \}$

Диалекти на BNF

Варианти на запис, различаващи се по начина на представяне (запис) на:

- Терминалите;
- Нетерминалите;
- Символът "произвежда";
- Разделителят между елементите;
- Нетерминалът и записът му;
- Завършване на правилото

Пример за алтернативно представяне- По международния стандарт ISO-14977:

- Терминалите са в "кавички";
- Нетерминалите се пишат естествено;
- Символът "произвежда" е =
- Запетая , е разделител между елементите а не-терминал може да се запише с повече от една дума;
- Всяко правило завършва с точка и запетая ;

Пример за if loop по ISO-14977 :

if loop = if part;

if part = "if" , "(" ,условен израз,")" , "{" ,тяло на блок, "}";

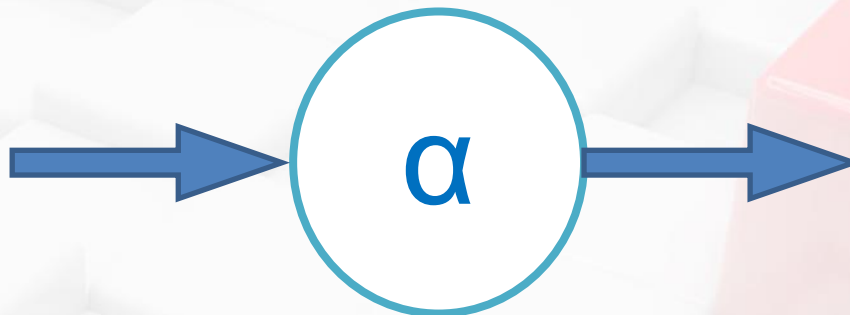
Допълнителна информация: <http://bnf-for-java.sourceforge.net/Dialects.html>

Синтактични графи

- ☐ Графично средство за представяне на правилата P .
- ☐ Чрез синтактичните графи се показва последователността от терминални и нетерминални символи, които се намират в дясната страна на правилото.
- ☐ Използват се графични изображения, в които се записват тези символи. Елементите се свързват със стрелки.
- ☐ Формата на изображението зависи от типа на символа: терминален или нетерминален.
- ☐ Използвайте кръгове или овали за терминалните елементи, и квадрати и правоъгълници за нетерминалните елементи.
- ☐ Свържете ги с линии и стрелки като следвате подходяща последователност.
- ☐ Могат да се съберат няколко правила в една диаграма.
- ☐ Използвайте цикли (loops) за да индикирате повторение.

Обозначения

$a \in \Sigma$



$u \in N$

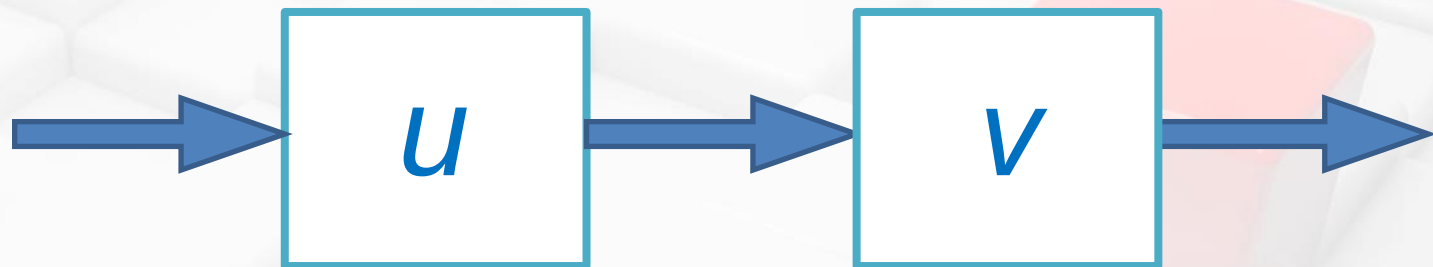


$\alpha \rightarrow \beta$

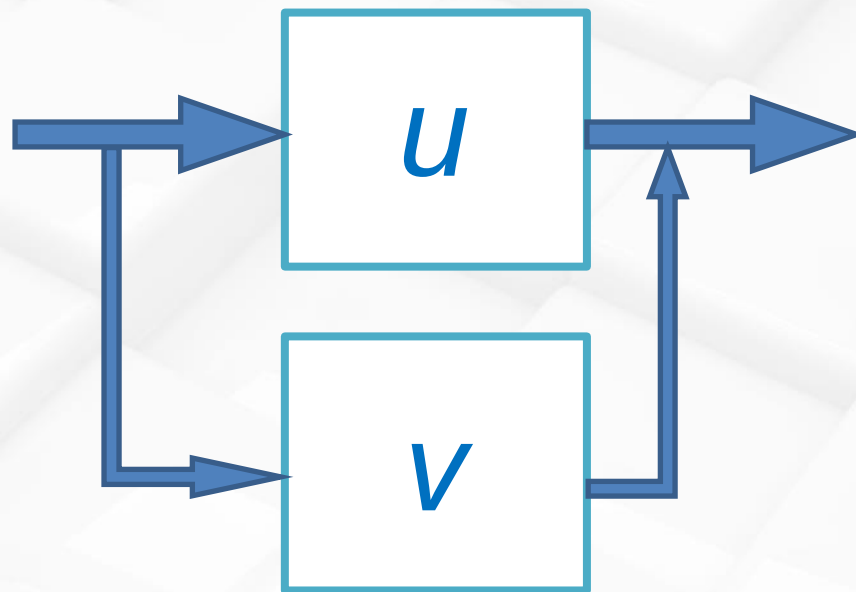


Обозначения

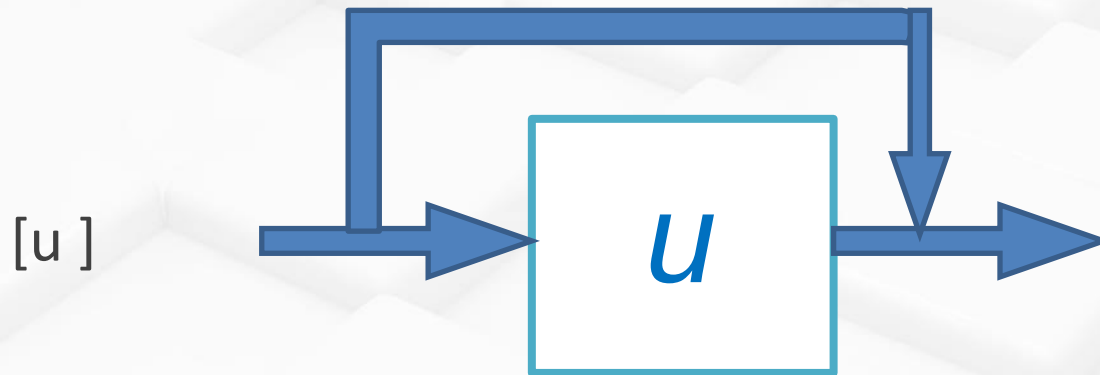
uv
конкатенация



$u \mid v$
альтернатива

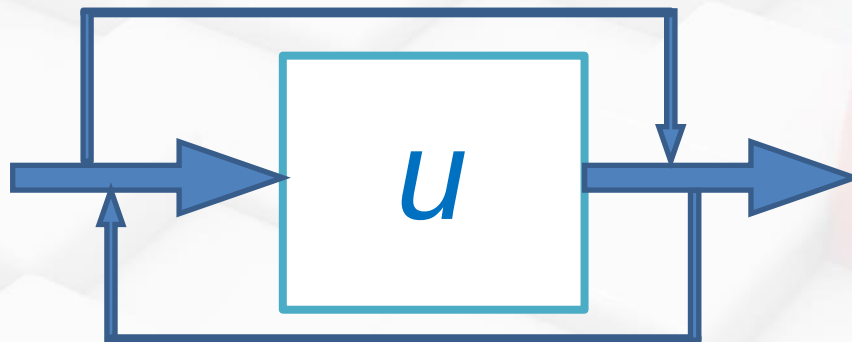


Обозначения

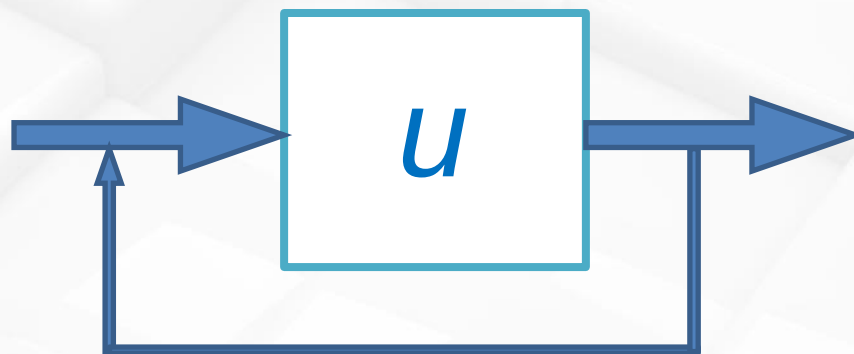


Обозначения

u^*
итерация

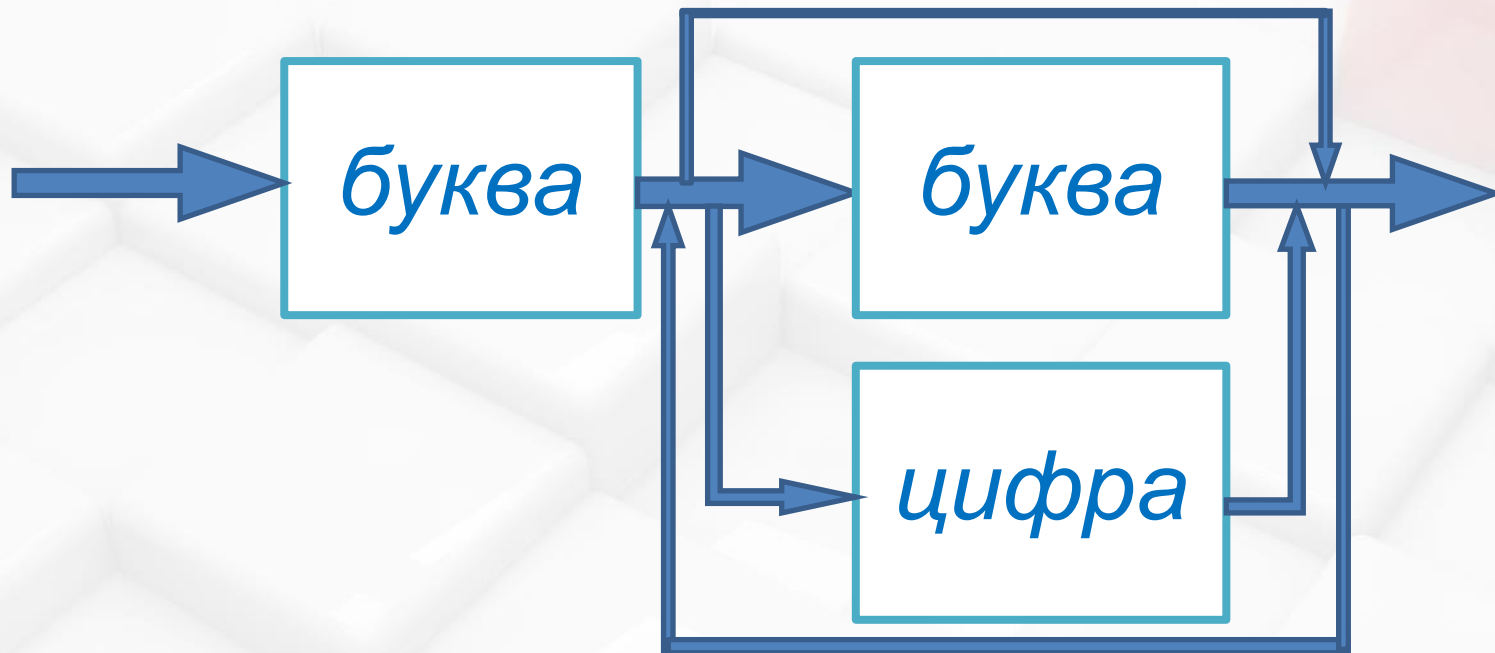


u^+
итерация без
празного
множества



Пример

Правило за описание на име на променлива



име => буква (буква | цифра)*

Името се съставя от задължителна начална буква и след нея може да има и други букви или цифри.

Пример

Всички елементи имат по един вход и по един изход.

Пример за означение на **if-else** израз.

